



ODG MOTOS

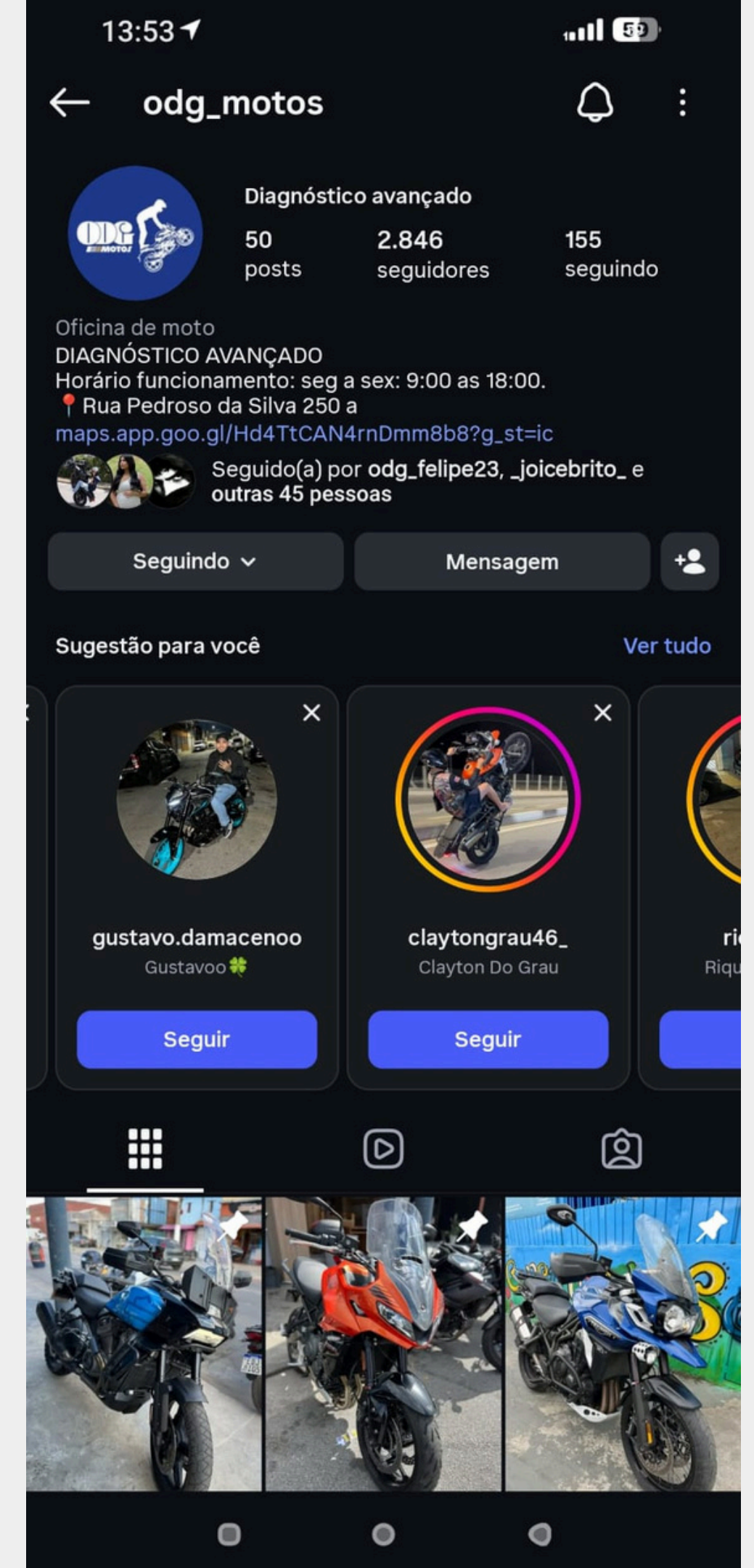
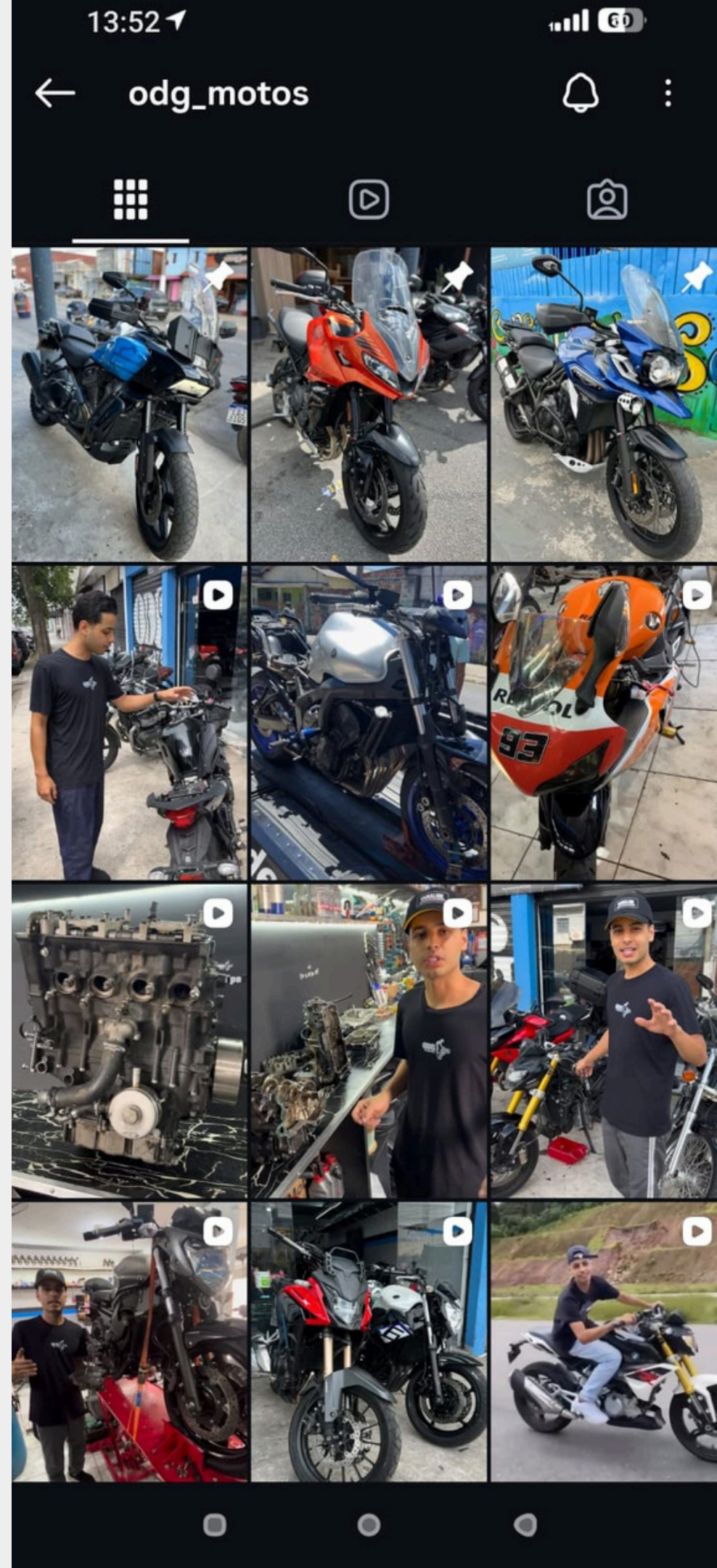
Projeto em Python

Criação de banco de dados

Sobre a empresa

A mais de 5 anos no ramo automotivo, a ODG motos busca melhoria em informações para rastreabilidade em serviços realizados pelo seus clientes.

Para isso realizamos uma breve consultoria na construção de um simples banco em python.



- Passo 1 :- construção do banco de dados:

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3
from PIL import Image, ImageTk
from tkinter import *
# --- Banco de Dados ---
def init_db():
    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS clientes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            email TEXT,
            telefone TEXT,
            endereco TEXT,
            servico TEXT,
            total REAL
        )
    ''')
    conn.commit()
    conn.close()
```

neste tópico estamos construindo a base do nosso banco para preenchimento.

- Passo 2 :- construção de armazenamento dos input:

```
def cadastrar_cliente():
    if not entry_nome.get():
        messagebox.showwarning("Erro", "O nome é obrigatório!")
        return

    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO clientes (nome, email, telefone, endereco, servico, total)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', (entry_nome.get(), entry_email.get(), entry_telefone.get(),
        entry_endereco.get(), entry_servico.get(), entry_total.get()))
    conn.commit()
    conn.close()
    limpar_campos()
    listar_clientes()
    messagebox.showinfo("Sucesso", "Cliente cadastrado com sucesso!")
```

neste tópico estamos construindo o armazenamento do nosso banco.

- Passo 3 :- tratativas do banco de dados:

```
def listar_clientes():
    for item in tree.get_children():
        tree.delete(item)

    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM clientes')
    for row in cursor.fetchall():
        tree.insert('', tk.END, values=row)
    conn.close()

def excluir_cliente():
    selected_item = tree.selection()
    if not selected_item:
        messagebox.showwarning("Erro", "Selecione um cliente para excluir!")
        return

    cliente_id = tree.item(selected_item)['values'][0]
    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()
    cursor.execute('DELETE FROM clientes WHERE id = ?', (cliente_id,))
    conn.commit()
    conn.close()
    listar_clientes()
    messagebox.showinfo("Sucesso", "Cliente excluído!")

def editar_cliente():
    selected_item = tree.selection()
    if not selected_item:
        messagebox.showwarning("Erro", "Selecione um cliente para editar!")
        return

    cliente_id = tree.item(selected_item)['values'][0]
    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()
    cursor.execute('''
        UPDATE clientes SET
        nome=?, email=?, telefone=?, endereco=?, servico=?, total=?
        WHERE id=?
    ''', (entry_nome.get(), entry_email.get(), entry_telefone.get(),
        entry_endereco.get(), entry_servico.get(), entry_total.get(), cliente_id))
    conn.commit()
    conn.close()
    limpar_campos()
    listar_clientes()
    messagebox.showinfo("Sucesso", "Dados atualizados!")
```

neste tópico seguimos com as tratativas de erro em preenchimento para o banco

- Passo 4 : - construção das informações para tabela

```
def preencher_campos(event):
    selected_item = tree.selection()
    if not selected_item:
        return

    cliente = tree.item(selected_item)['values']

    limpar_campos()
    entry_nome.insert(0, cliente[1])
    entry_email.insert(0, cliente[2])
    entry_telefone.insert(0, cliente[3])
    entry_endereco.insert(0, cliente[4])
    entry_servico.insert(0, cliente[5])
    entry_total.insert(0, cliente[6])

def limpar_campos():
    entry_nome.delete(0, tk.END)
    entry_email.delete(0, tk.END)
    entry_telefone.delete(0, tk.END)
    entry_endereco.delete(0, tk.END)
    entry_servico.delete(0, tk.END)
    entry_total.delete(0, tk.END)

def atualizar_banco():
    conn = sqlite3.connect('clientes.db')
    cursor = conn.cursor()

    try:
        cursor.execute("ALTER TABLE clientes ADD COLUMN servico TEXT")
        cursor.execute("ALTER TABLE clientes ADD COLUMN total REAL")
        print("Colunas adicionadas com sucesso!")
    except sqlite3.OperationalError:
        print("As colunas já existem ou a tabela não foi encontrada.")

    conn.commit()
    conn.close()
```

neste tópico estamos
construindo a estrutura
da tabela.

neste tópico estamos
construindo as janelas
e layont do nosso
sistema

- Passo 5 : - construção das janelas do sistema

```
tk.Label(frame_form, text="Nome:").grid(row=0, column=0, padx=5, pady=5)
entry_nome = tk.Entry(frame_form)
entry_nome.grid(row=0, column=1)

tk.Label(frame_form, text="E-mail:").grid(row=0, column=2, padx=5, pady=5)
entry_email = tk.Entry(frame_form)
entry_email.grid(row=0, column=3)

tk.Label(frame_form, text="Telefone:").grid(row=1, column=0, padx=5, pady=5)
entry_telefone = tk.Entry(frame_form)
entry_telefone.grid(row=1, column=1)

tk.Label(frame_form, text="Endereço:").grid(row=1, column=2, padx=5, pady=5)
entry_endereco = tk.Entry(frame_form)
entry_endereco.grid(row=1, column=3)

tk.Label(frame_form, text="Serviço:").grid(row=2, column=0, padx=5, pady=5)
entry_servico = tk.Entry(frame_form)
entry_servico.grid(row=2, column=1)

tk.Label(frame_form, text="Total (R$):").grid(row=2, column=2, padx=5, pady=5)
entry_total = tk.Entry(frame_form)
entry_total.grid(row=2, column=3)

frame_btns = tk.Frame(root)
frame_btns.pack(pady=10)

btn_cadastrar = tk.Button(frame_btns, text="Cadastrar", command=cadastrar_cliente, bg="#4CAF50", fg="white")
btn_cadastrar.grid(row=0, column=0, padx=10)

btn_editar = tk.Button(frame_btns, text="Atualizar", command=editar_cliente, bg="#2196F3", fg="white")
btn_editar.grid(row=0, column=1, padx=10)

btn_excluir = tk.Button(frame_btns, text="Excluir", command=excluir_cliente, bg="#f44336", fg="white")
btn_excluir.grid(row=0, column=2, padx=10)

btn_limpar = tk.Button(frame_btns, text="Limpar Campos", command=limpar_campos)
btn_limpar.grid(row=0, column=3, padx=10)

tree = ttk.Treeview(root, columns=("ID", "Nome", "Email", "Telefone", "Endereço", "Serviço", "Total"), show='headings')
tree.heading("ID", text="ID")
tree.heading("Nome", text="Nome")
tree.heading("Email", text="Email")
tree.heading("Telefone", text="Telefone")
tree.heading("Endereço", text="Endereço")
tree.heading("Serviço", text="Serviço")
```

Resumo de cadastro em banco de dados para apuração de serviços realizados, para a identificação de clientes, veículos, serviços e valores cobrados pelos serviços.

ODG MOTOS

ODG MOTOS

Nome:

E-mail:

Telefone:

Endereço:

Serviço:

Total (R\$):

Cadastrar

Atualizar

Excluir

Limpar Campos

ID	Nome	Email	Telefone	Endereço	Serviço	Total
4	felipe	felipeOdg	11944017215	rua pedroso da silva	revisão	1000.0